Practical 3

#Problem solving by searching (Informed Search)

```
import heapq
graph = {
    'Jatin':{'Zaidi': 75,   'Abhinash': 140, 'Sarfaraz': 118},
    'Zaidi': {'Jatin': 75,  'Ashraf': 71},
    'Ashraf':{'Zaidi':71, 'Abhinash':151,},
    'Abhinash': {'Jatin':140, 'Ashraf':151, 'Sachin':118},
    'Sarfaraz':{'Jatin':118, 'Sachin': 111},
    'Sachin':{'Abhinash':99, 'Sarfaraz': 111}
    }
heuristic = {'Jatin':180, 'Zaidi':250, 'Ashraf':200,
             'Abhinash':80, 'Sarfaraz':90, 'Sachin':0
    }
def a_star_search(graph, start, goal,h):
    frontier = [(h[start], 0, start,[start])]
    visited = set()

    while frontier:
        f_cost, g_cost, node, path = heapq.heappop(frontier)

        if node == goal:
            return path,g_cost

        if node in visited:
            continue
        visited.add(node)

        for neighbour, weight in graph.get(node,{}).items():
            if neighbour not in visited:
```

```
                new_g = g_cost + weight
                new_f = new_g + h.get(neighbour,0)
                heapq.heappush(frontier,(new_f, new_g, neighbour, path +
[neighbour]))
    return None, float('inf')
if __name__=="__main__":
   path, cost = a_star_search(graph, 'Jatin', 'Sachin', heuristic)
   print("Least-cost path from Jatin to Sachin(A*);")
   print("->".join(path))
   print("Total cost:", cost)
```

Output:-

```
Least-cost path from Jatin to Sachin(A*);
Jatin->Sarfaraz->Sachin
Total cost: 229
```

```
# Practical No: 5
# Naive Bayes Classification
# Using a small dataset (Play Tennis dataset)

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import CategoricalNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset
data = {
    'Outlook': [
        'Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
        'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny',
```

```
        'Overcast', 'Overcast', 'Rain'
    ],
    'Temperature': [
        'Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
        'Cool', 'Mild', 'Cool', 'Mild', 'Mild',
        'Mild', 'Hot', 'Mild'
    ],
    'Humidity': [
        'High', 'High', 'High', 'High', 'Normal', 'Normal',
        'Normal', 'High', 'Normal', 'Normal', 'Normal',
        'High', 'Normal', 'High'
    ],
    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
        'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
        'Strong', 'Weak', 'Strong'
    ],
    'PlayTennis': [
        'No', 'No', 'Yes', 'Yes', 'Yes', 'No',
        'Yes', 'No', 'Yes', 'Yes', 'Yes',
        'Yes', 'Yes', 'No'
    ]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Dataset:\n")
print(df)
```

```
# Encode categorical data
encoders = {}
df_encoded = df.copy()

for col in df.columns:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df[col])
    encoders[col] = le

# Features and target
X = df_encoded.drop('PlayTennis', axis=1)
y = df_encoded['PlayTennis']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train Naive Bayes model
model = CategoricalNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("\nPredicted Values:", y_pred)
print("Actual Values   :", y_test.values)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

print("\nTest Set Accuracy:", accuracy)

```
C:\Users\Umair Khan>python -m pip install pandas
```

```
C:\Users\Umair Khan>pip install scikit-learn
```

Output:-

```
Dataset:

     Outlook Temperature Humidity    Wind PlayTennis
0      Sunny         Hot     High    Weak         No
1      Sunny         Hot     High  Strong         No
2   Overcast         Hot     High    Weak        Yes
3       Rain        Mild     High    Weak        Yes
4       Rain        Cool   Normal    Weak        Yes
5       Rain        Cool   Normal  Strong         No
6   Overcast        Cool   Normal  Strong        Yes
7      Sunny        Mild     High    Weak         No
8      Sunny        Cool   Normal    Weak        Yes
9       Rain        Mild   Normal    Weak        Yes
10     Sunny        Mild   Normal  Strong        Yes
11  Overcast        Mild     High  Strong        Yes
12  Overcast         Hot   Normal    Weak        Yes
13      Rain        Mild     High  Strong         No

Predicted Values: [1 0 0 1 1]
Actual Values   : [1 1 0 1 0]

Test Set Accuracy: 0.6
```

Practical No: 6

Using a simple example (Burglary-Alarm), create a Bayesian Network using pgmpy

from pgmpy.models import DiscreteBayesianNetwork

from pgmpy.factors.discrete import TabularCPD

from pgmpy.inference import VariableElimination

Create Bayesian Network

```
model = DiscreteBayesianNetwork([
    ('Burglary', 'Alarm'),
    ('Earthquake', 'Alarm')
])

# Define CPDs
cpd_burglary = TabularCPD(
    variable='Burglary',
    variable_card=2,
    values=[[0.999], [0.001]]
)

cpd_earthquake = TabularCPD(
    variable='Earthquake',
    variable_card=2,
    values=[[0.998], [0.002]]
)

cpd_alarm = TabularCPD(
    variable='Alarm',
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05],  # Alarm = False
        [0.001, 0.29, 0.94, 0.95]   # Alarm = True
    ],
    evidence=['Burglary', 'Earthquake'],
    evidence_card=[2, 2]
)

# Add CPDs to the model
```

```
model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm)

# Check if model is valid
assert model.check_model()

# Perform inference
infer = VariableElimination(model)

# Query P(Burglary | Alarm=True)
posterior = infer.query(
    variables=['Burglary'],
    evidence={'Alarm': 1}
)

print("P(Burglary | Alarm = True):")
print(posterior)

# Probability of Burglary=True given Alarm=True
p_burglary_true = posterior.get_value(Burglary=1)
print(f"\nP(Burglary = True | Alarm = True) = {p_burglary_true:.4f}")
```

```
C:\Users\Umair Khan>python -m pip install pgmpy
```

Output:-

```
Warning (from warnings module):
  File "C:\Users\Umair Khan\AppData\Local\Progra
    from .StructureScore import (
FutureWarning: `pgmpy.estimators.StructureScore
P(Burglary | Alarm = True):
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.6264 |
+-------------+-----------------+
| Burglary(1) |          0.3736 |
+-------------+-----------------+

P(Burglary = True | Alarm = True) = 0.3736
```

practical 7

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output:-

```
Accuracy: 1.0
```

practical 8

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
data = load_diabetes()
x,y = data.data, data.target

x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Feature names:", data.feature_names)
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:",mse)
print("R2 Score:",r2)
```

```
C:\Users\Umair Khan>python -m pip install notebook
```

Output:-

```
Feature names: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
Coefficients: [  37.90402135 -241.96436231  542.42875852  347.70384391 -931.48884588
  518.06227698  163.41998299  275.31790158  736.1988589    48.67065743]
Intercept: 151.34560453985995
Mean Squared Error: 2900.1936284934804
R2 Score: 0.4526027629719197
```

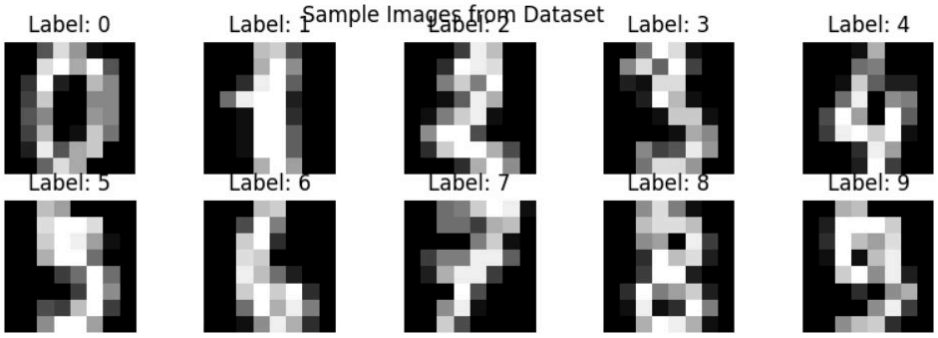
Practical No: 9

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

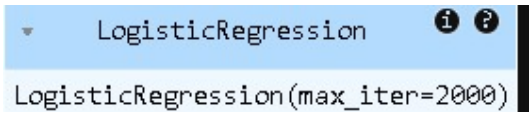
digits = load_digits()
x, y = digits.data, digits.target
images= digits.images

plt.figure(figsize=(10,3))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images[i], cmap= 'gray')
    plt.title(f"Label: {y[i]}")
    plt.axis('off')
plt.suptitle("Sample Images from Dataset")
plt.show()
```




```
TARGET_DIGIT = 5
y_binary = (y == TARGET_DIGIT).astype(int)
x_train, x_test, y_train, y_test,img_train,img_test = train_test_split(
    x,y_binary,images,test_size=0.3,random_state=42, stratify=y_binary).
```

```
model =  LogisticRegression(max_iter=2000)
model.fit(x_train,y_train)
```




```
y_pred = model.predict(x_test)

print(f"Binary Classification Task: Digit == {TARGET_DIGIT} vs Not {TARGET_DIGIT}")
print(f"Accuracy:", accuracy_score(y_test, y_pred))
print("Classification report:", classification_report(y_test, y_pred))
```

```
Binary Classification Task: Digit == 5 vs Not 5
Accuracy: 0.9944444444444445
Classification report:               precision    recall  f1-score   support

           0       1.00      1.00      1.00       485
           1       0.98      0.96      0.97        55

    accuracy                           0.99       540
   macro avg       0.99      0.98      0.98       540
weighted avg       0.99      0.99      0.99       540
```

Practical No: 10

#Implement k-fold cross-validation on a classification model

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
x, y = iris.data, iris.target

model = DecisionTreeClassifier(random_state=42)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, x, y, cv=kf)

print("Accuracy for each fold:", np.round(scores,4))
print("Mean Accuracy:", scores.mean())
print("Standard Deviation:", scores.std())
```

Output:-

```
Accuracy for each fold: [1.     0.9667 0.9333 0.9333 0.9333]
Mean Accuracy: 0.9533333333333335
Standard Deviation: 0.02666666666666666
```

Practical No: 11

(CMD: python -m -version)

#Implement a Decision Tree classifier using the Iris or any relevant dastaset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

iris = load_iris()
x, y = iris.data, iris.target

x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.3, random_state=42)

model = DecisionTreeClassifier(criterion='entropy',max_depth=3,random_state=42)
model.fit(x_train,y_train)
y_pred = model.predict(x_test)

print("Test Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

print("\nDecision Tree Rules:\n")
print(export_text(model, feature_names=list(iris.feature_names)))
```

```
Test Accuracy: 0.9777777777777777

Classification Report:

              precision    recall  f1-score   support

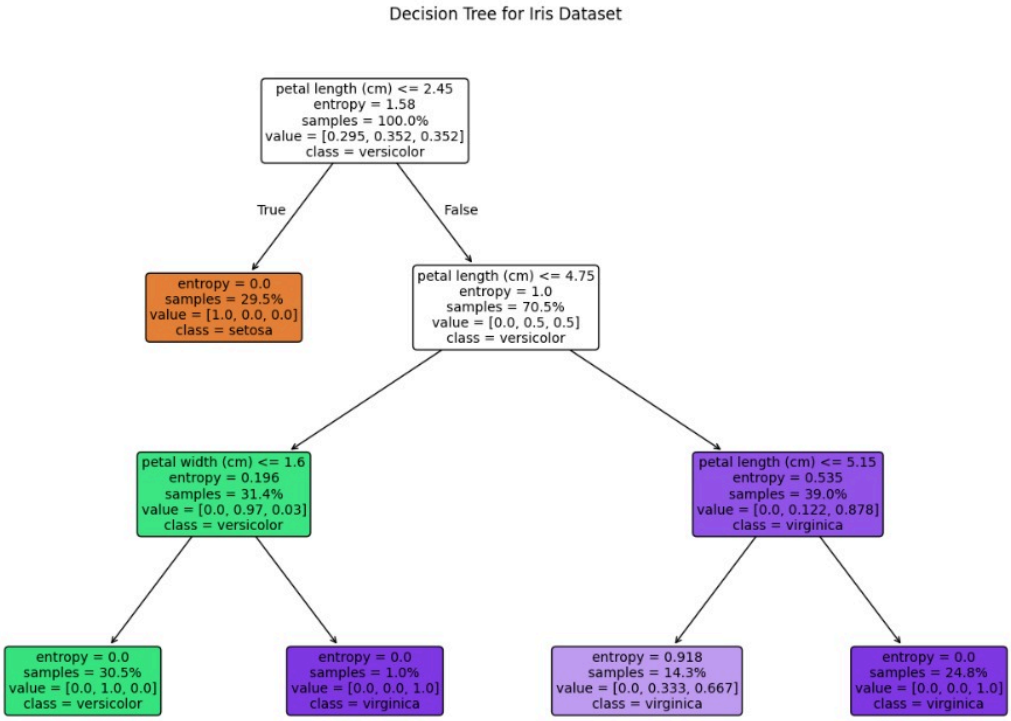
      setosa       1.00      1.00      1.00        19
  versicolor       1.00      0.92      0.96        13
   virginica       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
```

```
Decision Tree Rules:

|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.60
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.60
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal length (cm) <= 5.15
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  5.15
|   |   |   |--- class: 2
```

```
plt.figure(figsize=(15, 10))
plot_tree(model,
feature_names=iris.feature_names,class_names=iris.target_names,
        filled=True,rounded=True,fontsize=10,proportion=True)
plt.title("Decision Tree for Iris Dataset")
plt.show()
```



Practical No: 12

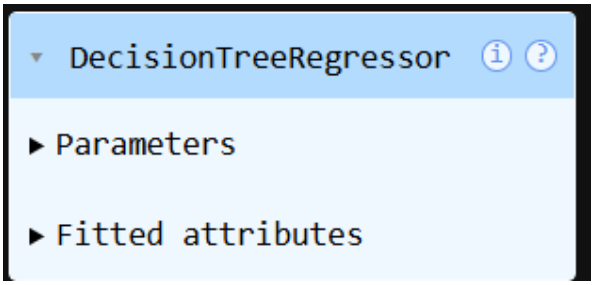
#Implement Decision Tree regression on a regression dataset

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_squared_error

data = load_diabetes()
x,y = data.data, data.target

x_train, x_test, y_train, y_test= train_test_split(x,y, test_size=0.2, random_state=42)

model =  DecisionTreeRegressor(max_depth=4, random_state=42)
model.fit(x_train, y_train)
```




```
y_pred  = model.predict(x_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test,y_pred))
```

```
Mean Squared Error: 3568.9652613232283
R^2 Score: 0.3263754171777956
```

```
print("\n Feature Importances:")
```

```
for name, importance in zip(data.feature_names, model.feature_importances_):
    print(f" {name}: {importance:.4f}")
```

```
Feature Importances:
age: 0.0168
sex: 0.0000
bmi: 0.6292
bp: 0.0000
s1: 0.0115
s2: 0.0189
s3: 0.0192
s4: 0.0251
s5: 0.2130
s6: 0.0662
```

Practical No: 13

#Implement k_Neares Neighbours (kNN) classifier

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

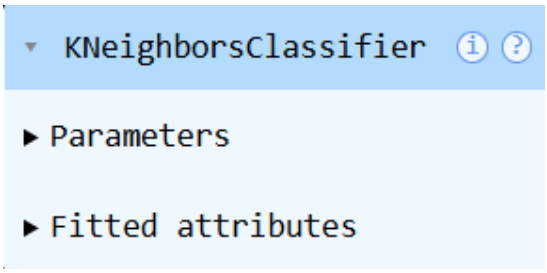
iris = load_iris()
x , y = iris.data, iris.target

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size =0.3,random_state =
42)

scaler = StandardScaler()
```

```
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(x_train_scaled, y_train)
```




```
y_pred = model.predict(x_test_scaled)

print("Accuracy Score:", accuracy_score(y_test,y_pred))
print("Classifier report:", classification_report(y_test,y_pred))
```

```
Accuracy Score: 1.0
Classifier report:              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Practice No: 14

```
#Implement multiclass classification using Logistic Regression or Decision Tree

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
x , y = iris.data, iris.target

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size =0.3,random_state =
42)

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

lr = LogisticRegression(max_iter=200)
lr.fit(x_train_scaled, y_train)
y_pred = lr.predict(x_test_scaled)

print("Accuracy Score:", accuracy_score(y_test,y_pred))
print("Classifier report:", classification_report(y_test,y_pred,
target_names=iris.target_names))
```

```
Accuracy Score: 1.0
Classifier report:              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)

tree.fit(x_train, y_train)

y_pred_tree = tree.predict(x_test)


print("Accuracy Score:", accuracy_score(y_test,y_pred_tree))

print("Classifier report:", classification_report(y_test,y_pred_tree,
target_names=iris.target_names))
```

```
Accuracy Score: 1.0
Classifier report:              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```